MarketFlow — Parte 1/3: Arquitectura maestra (System Data Sheet v1.0)~h j

Oct 10, 2026 · @jefferson

1. Evaluación y alcance del proyecto

MarketFlow es viable para un junior solo si se construye en tres fases con puertas de calidad; el esfuerzo orientativo es de 160–220 h (estimación de planificación, no medida). Esta etapa es solo de arquitectura: no incluye código.

Conocimientos que hay que adquirir: SQL con transacciones e índices, HTTP/REST, JWT y cookies, Fastify (plugins, hooks y encapsulación), Prisma, Redis (estructuras y scripts Lua), BullMQ, Docker y pruebas de concurrencia.

Qué entender antes de implementar: `UPDATE` condicional y atomicidad en PostgreSQL, idempotencia, HMAC sobre cuerpo crudo, expiración por TTL y por trabajos, y por qué Lua es atómico solo dentro de Redis.

Alcance	Funcionalidades
MVP imprescindible	Auth con roles, catálogo y gestión de productos del vendedor por API (el panel visual se aplaza), carrito en cliente, checkout con reserva, pago mock con webhook idempotente, expiración de reservas, pruebas y despliegue
Avanzadas	Reservas atómicas con Redis y Lua, reconciliación, colas BullMQ, comprobación de refresh en Redis, pruebas de carga con k6
Aplazables	Stripe como dependencia obligatoria, reembolsos parciales, multimoneda, envíos e impuestos, subida de imágenes, 2FA, i18n, email real, importación de Open Food Facts hasta después de la Fase 1

Riesgos técnicos principales: divergencia entre Redis y PostgreSQL; workers que se duermen en planes gratuitos; webhooks duplicados o tardíos; cookies entre dominios distintos; crecimiento del alcance.

Fase	Objetivo	Puerta de salida
1. MVP funcional	Compra de extremo a extremo con 22 endpoints núcleo e inventario seguro solo con PostgreSQL (reservas con tablas propias y límites de tasa en memoria)	G1: 50 checkouts simultáneos sobre stock 10 crean exactamente 10 reservas; invariantes intactos
2. Concurrencia e inventario	Redis y Lua como control de admisión, reconciliación y pruebas de caos	G2: 200 checkouts sobre stock 50 dan exactamente 50 éxitos, incluso con un reinicio de Redis
3. Preparación profesional	BullMQ, observabilidad, pruebas de carga, CI/CD y despliegue	G3: expiración procesada con p95 ⩽ 30 s y checklist de producción completo

Nota honesta: el `UPDATE` condicional de PostgreSQL ya evita la sobreventa. Redis no es necesario para la corrección; se añade como control de admisión rápido y como ejercicio de consistencia entre dos sistemas.

Fase 1: alcance y orden de implementación

La Fase 1 se limita a 22 endpoints núcleo; los otros 19 se completan antes de G3. Cada tarea se da por terminada solo cuando pasa su prueba.

Imprescindible para una compra funcional (22 endpoints):

- **Auth y perfil (6):** `POST /auth/register`, `POST /auth/login`, `POST /auth/refresh`, `POST /auth/logout`, `GET /users/me`, `POST /seller-profile`.
- **Catálogo público (3):** `GET /products`, `GET /products/:slug`, `GET /categories`.
- **Vendedor (4):** `GET /seller/products`, `POST /seller/products`, `PATCH /seller/products/:id`, `POST /seller/inventory/:productId/adjustments`.
- **Pedidos (4):** `POST /checkout`, `GET /orders`, `GET /orders/:id`, `POST /orders/:id/cancel`.
- **Pagos (3):** `POST /orders/:id/payments`, `POST /mock-gateway/payments/:id/confirm`, `POST /webhooks/payments/:provider`.
- **Operación (2):** `GET /health/live`, `GET /health/ready`.

No se aplaza nunca (seguridad e integridad): hash Argon2id, rotación de refresh con detección de reutilización, validación Zod, comprobación de roles y de propiedad, CHECK en PostgreSQL, `UPDATE` condicional con libro de movimientos, idempotencia del

checkout, firma HMAC con bandeja de entrada única, barrido de expiración y límites de tasa en memoria.

Aplazado hasta antes de G3 (19 endpoints): gestión de sesiones y `logout-all` (3), `PATCH /users/me` (1), lectura y edición del perfil de vendedor (2), CRUD de categorías (3), `GET /categories/:slug` (1), ver y borrar un producto propio (2), disponibilidad, listado y movimientos de inventario (3), cotización (1), pedidos del vendedor y cumplimiento (2) y `GET /payments/:id` (1).

Otros aplazamientos: tabla y escrituras de `AuditLog` (en la Fase 1 solo hay logs de la aplicación), panel visual del vendedor, importación de Open Food Facts, Redis, BullMQ y Stripe.

N.º	Tarea	Depende de	Avanzas cuando…
1	Repositorio, Docker con PostgreSQL, lint y CI con una prueba trivial	—	`docker compose ps` sale `healthy` y la CI está en verde
2	API Fastify mínima: configuración validada con Zod, health checks, manejador de errores y logs	1	`/health/live` devuelve 200 y, sin una variable obligatoria, el arranque falla con un mensaje claro
3	Prisma y migraciones de auth y catálogo, con los CHECK en SQL	2	`migrate status` está al día y un dato inválido falla
4	Registro, login, `GET /users/me`, `POST /seller-profile` (perfil `APPROVED`) con Argon2id, JWT y hooks de rol	3	401 sin token, 403 por rol y la contraseña no se guarda en claro
5	Refresh con rotación, logout y detección de reutilización	4	Un token reutilizado revoca su familia; la cookie es `HttpOnly` y `SameSite=Strict`
6	Límites de tasa en memoria (login, registro, checkout)	5	El 6.º login en un minuto devuelve 429
7	Importación de DummyJSON y FakeStoreAPI y catálogo público con paginación por cursor	3	Importar dos veces deja los mismos recuentos, funciona sin Internet y la paginación no repite ni omite

N.º	Tarea	Depende de	Avanzas cuando…
8	Productos del vendedor; migraciones de inventario y pedidos; `UPDATE` condicional, libro de movimientos y ajuste de stock	7	Reservar más de lo disponible afecta 0 filas y el script de invariantes I1–I3 no da discrepancias
9	Checkout idempotente con reserva `ACTIVE`, pedidos, cancelación y barrido de expiración (al arrancar y cada 30 s)	6, 8	Doble envío da un solo pedido, cancelar libera stock, un pedido vencido libera en ⩽ 2 min y 50 checkouts simultáneos sobre stock 10 dan exactamente 10 reservas
10	Pago mock y webhook firmado e idempotente, entregado dentro del propio proceso (migración de pagos)	9	Firma inválida da 401, el mismo evento ×10 tiene un solo efecto, un pago tardío acaba en `REFUNDED` y un pago correcto deja el pedido `PAID` con el stock consumido
11	Frontend mínimo: catálogo, ficha, login, carrito, checkout, pago mock con cuenta atrás y «Mis pedidos»	5, 7, 10	Un E2E de compra completa pasa, no hay tokens en `localStorage` y la cookie es `HttpOnly`
12	Puerta G1: repetir 10 veces la prueba concurrente, las invariantes y el E2E	1–11	Todo en verde sin un solo fallo

2. Arquitectura y stack seleccionados

La arquitectura es un monolito modular Fastify con un proceso worker opcional; PostgreSQL es la fuente de verdad y Redis actúa como control de admisión y cola. No hay microservicios. Las versiones exactas se fijan al empezar la implementación.

Capa	Elección	Por qué	Alternativa descartada
Frontend	Next.js App Router, React (JavaScript ES2022+ con JSX), Tailwind CSS, shadcn/ui	SSR/ISR para catálogo y accesibilidad por defecto	Vite + React Router

Capa	Elección	Por qué	Alternativa descartada
Backend	**Fastify con plugins por módulo**	Cada módulo es un plugin encapsulado de Fastify, lo que refuerza las fronteras; funciona con JavaScript puro, sin decoradores ni transpilado; BullMQ se usa directamente	NestJS (pensado para TypeScript; en JavaScript exige Babel con decoradores e inyección explícita)
Datos	PostgreSQL + Prisma	Transacciones, CHECK, índices parciales; migraciones y tipado	Drizzle o Kysely
Redis y colas	Redis + BullMQ	Scripts Lua, límites de tasa, trabajos retrasados con reintentos	pg-boss (colas sobre PostgreSQL)
Pruebas	Vitest, Supertest, Testcontainers, Playwright, k6	Unitarias, integración, E2E y carga	Jest
Entrega	pnpm workspaces, Docker, GitHub Actions	Reproducibilidad	Turborepo (opcional)

El monorepo tiene `apps/web`, `apps/api` (con un segundo punto de entrada para el worker) y `packages/` para base de datos, contratos Zod compartidos y configuración.

Lenguaje: JavaScript moderno (ES2022+) con ES modules (`"type": "module"`) en frontend y backend; `.js` en la API y el worker, `.jsx` en componentes de React y `.mjs` solo cuando haya que forzar ESM (scripts y configuración). No hay TypeScript; la validación de contratos ocurre en tiempo de ejecución con Zod y se puede documentar con JSDoc.

Convenciones de JavaScript que heredan las Partes 2 y 3:

- **Calidad:** sin `tsconfig` ni `typecheck`; la integración continua ejecuta lint y pruebas. Un `jsconfig.json` y JSDoc son opcionales y solo ayudan al editor.
- **Fastify:** un plugin de rutas por controller y un módulo o clase por service; los services compartidos se registran con `fastify.decorate`; la autenticación y los roles son hooks `preHandler`; los errores pasan por `setErrorHandler`; el cuerpo crudo de los webhooks se captura con un parser de contenido propio.

- **Validación:** `packages/contracts` exporta solo esquemas Zod; se validan con `parse` en un hook o en el handler, sin proveedores de tipos de Zod para Fastify.
- **Tareas diferidas:** `node-cron` en la Fase 1 y BullMQ desde la Fase 3.
- **Pruebas y scripts:** Vitest con `fastify.inject`; scripts de importación y utilidades en `.mjs` ejecutados con `node`.
- **Datos:** Prisma Client y BullMQ se usan desde JavaScript; el `UPDATE` condicional de inventario va con `$executeRaw`.

Evaluación de plataformas gratuitas

No se asume que los planes gratuitos sean estables ni compatibles entre sí; cada límite se verifica con la documentación vigente antes de desplegar.

Plataforma	Rol previsto	Restricciones a verificar
Vercel	Frontend Next.js	Condiciones del plan gratuito (uso no comercial), límites de ancho de banda y funciones, `rewrites` hacia un dominio externo
Render	API Fastify y, si es posible, el worker	Suspensión por inactividad y arranque en frío, existencia de worker persistente gratuito, horas mensuales
Supabase	PostgreSQL	Pausa por inactividad, conexiones máximas, pooler en modo transacción, copias de seguridad
Upstash Redis	Redis para Lua, límites y BullMQ	Límite de comandos (el polling de BullMQ consume), tamaño, soporte de `EVAL`, política `noeviction`, conexión TCP

Restricciones serverless: BullMQ necesita un proceso Node persistente y una conexión TCP a Redis, por lo que la API y el worker no pueden vivir en funciones serverless de Vercel. El worker tiene tres modos: `embedded` (dentro de la API, opción gratuita; si la instancia duerme, las expiraciones se retrasan y las recupera un barrido que también se ejecuta al arrancar la API; en la Fase 1 no hay BullMQ y ese barrido corre dentro de la API con node-cron), `separate` (servicio propio, si el plan lo permite) y `off`.

Cookies: frontend y API están en dominios distintos y una cookie `SameSite=Strict` no viajaría entre ellos. Por eso el navegador llama siempre a `/api/*` en el origen del frontend y Next.js lo reescribe hacia Render conservando el prefijo (/api/:path* hacia la URL de la

API más /api/:path*); antes de desplegar hay que comprobar que ese proxy tolera el arranque en frío de Render.

3. Diagramas de componentes y datos

Componentes

```
flowchart TB
  U[Navegador] -->|HTTPS| W[Next.js en Vercel]
  W -->|rewrite /api/* mismo origen| A[API Fastify en Render]
  A -->|Prisma via pooler| P[(PostgreSQL Supabase)]
  A -->|ioredis TLS| R[(Redis Upstash)]
  K[Worker BullMQ] -->|consume trabajos| R
  K -->|Prisma| P
  A ---|mismo codigo otro entrypoint| K
  A -->|crea pago| G[Pasarela mock interna]
  G -.->|webhook firmado| A
  S[Stripe Test Mode opcional] -.->|webhook firmado| A
  A -.->|Checkout alojado| S
  C[CLI de importacion] -->|seed local| P
  X[DummyJSON FakeStoreAPI Open Food Facts] -.->|solo importacion| C
```

Las líneas discontinuas son flujos opcionales o puntuales: las APIs de datos solo alimentan el CLI de importación y Stripe es opcional.

Datos

```
erDiagram
  USERS ||--o{ USER_ROLES : has
  USERS ||--o| SELLER_PROFILES : owns
  USERS ||--o{ REFRESH_TOKEN_SESSIONS : opens
  USERS ||--o{ ORDERS : places
  USERS ||--o{ IDEMPOTENCY_KEYS : sends
  USERS ||--o{ AUDIT_LOGS : acts
  SELLER_PROFILES ||--o{ PRODUCTS : sells
  SELLER_PROFILES ||--o{ ORDER_ITEMS : fulfills
  CATEGORIES ||--o{ CATEGORIES : parent_of
  CATEGORIES ||--o{ PRODUCTS : classifies
  PRODUCTS ||--o| INVENTORY : has
  INVENTORY ||--o{ INVENTORY_MOVEMENTS : logs
  PRODUCTS ||--o{ ORDER_ITEMS : snapshot_of
  PRODUCTS ||--o{ RESERVATION_ITEMS : held_in
  ORDERS ||--|{ ORDER_ITEMS : contains
  ORDERS ||--o| RESERVATIONS : reserves
```

```
  ORDERS ||--o{ PAYMENTS : paid_by
  ORDERS ||--o{ INVENTORY_MOVEMENTS : causes
  RESERVATIONS ||--|{ RESERVATION_ITEMS : contains
  PAYMENTS ||--o{ WEBHOOK_EVENTS : notified_by
```

4. Modelo de datos conceptual

El modelo tiene 16 entidades. Además de las pedidas se añaden `Inventory` (stock autoritativo aislado del catálogo), `InventoryMovement` (libro mayor que permite verificar el stock), `ReservationItem` (reserva multiproducto) e `IdempotencyKey` (reintentos seguros de checkout y pago). No hay tabla `Role`: los roles son un enum en `UserRole`, porque los permisos son fijos en código.

Entidad	Campos principales	Relaciones	Índices y restricciones
`User`	`email`, `passwordHash`, `displayName`, `status`, `deletedAt`	1:N `UserRole`, `RefreshTokenSession`, `Order`; 1:0..1 `SellerProfile`	`email` único y en minúsculas
`UserRole`	`userId`, `role`	N:1 `User`	PK `(userId, role)`
`SellerProfile`	`userId`, `storeName`, `storeSlug`, `status`	1:1 `User`; 1:N `Product`, `OrderItem`	`userId` y `storeSlug` únicos
`RefreshTokenSession`	`userId`, `familyId`, `tokenHash`, `expiresAt`, `usedAt`, `revokedAt`	N:1 `User`	`tokenHash` único; `(userId, revokedAt)`; `familyId`
`Category`	`parentId`, `name`, `slug`	Árbol de dos niveles; 1:N `Product`	`slug` único
`Product`	`sellerId`, `categoryId`, `title`, `slug`, `priceCents`, `currency`, `imageUrls`, `status`, `source`, `externalId`, `deletedAt`	N:1 `SellerProfile`, `Category`; 1:0..1 `Inventory`	`slug` único; `(source, externalId)` único; `(categoryId, status, createdAt)`; búsqueda trigram sobre `title`
`Inventory`	`productId`, `onHand`, `reserved`	1:1 `Product`	CHECK `0 <= reserved <= onHand`
`InventoryMovement`	`productId`, `type`, `delta`, `onHandAfter`, `orderId`	N:1 `Inventory`, `Order`	Solo inserción; `(productId, createdAt)`

Entidad	Campos principales	Relaciones	Índices y restricciones
`Reservation`	`orderId`, `status`, `expiresAt`, `resolvedAt`	1:1 `Order`; 1:N `ReservationItem`	`orderId` único; `(status, expiresAt)`
`ReservationItem`	`reservationId`, `productId`, `quantity`	N:1 `Reservation`, `Product`	PK `(reservationId, productId)`; cantidad > 0
`Order`	`orderNumber`, `customerId`, `status`, `subtotalCents`, `shippingCents`, `totalCents`, `shippingAddress`, `expiresAt`	N:1 `User`; 1:N `OrderItem`, `Payment`	`(customerId, createdAt)`; `(status, expiresAt)`; CHECK de totales
`OrderItem`	`orderId`, `productId`, `sellerId`, `titleSnapshot`, `unitPriceCents`, `quantity`, `lineTotalCents`, `fulfillmentStatus`	N:1 `Order`, `Product`, `SellerProfile`	`(orderId, productId)` único; `(sellerId, fulfillmentStatus)`; CHECK de línea
`Payment`	`orderId`, `provider`, `providerPaymentId`, `status`, `amountCents`	N:1 `Order`; 1:N `WebhookEvent`	`(provider, providerPaymentId)` único; un pago vivo por pedido (índice parcial)
`WebhookEvent`	`provider`, `eventId`, `type`, `payload`, `status`, `attempts`	N:1 `Payment`	`(provider, eventId)` único
`IdempotencyKey`	`userId`, `scope`, `key`, `requestHash`, `responseBody`, `expiresAt`	N:1 `User`	`(userId, scope, key)` único
`AuditLog`	`actorType`, `actorId`, `action`, `entityType`, `entityId`, `metadata`, `occurredAt`	N:1 `User` (opcional)	Solo inserción; `(entityType, entityId, occurredAt)`

Representación de datos

- **Dinero:** enteros en céntimos y una sola moneda (EUR); nunca `float`.
- **Cantidades:** enteros mayores que 0; límites de negocio propuestos de 10 unidades por línea y 20 líneas por checkout.
- **Fechas:** `timestamptz` en UTC; `expiresAt` lo calcula el reloj de PostgreSQL, no el cliente.

- **Estados:** enums nativos de PostgreSQL; las transiciones se aplican con compare-and-set sobre el estado esperado. Pedido: `PENDING_PAYMENT`, `PAID`, `CANCELLED`, `EXPIRED`, `COMPLETED`. Pago: `PENDING`, `REQUIRES_ACTION`, `SUCCEEDED`, `FAILED`, `CANCELLED`, `REFUNDED`. Reserva: `ACTIVE`, `CONSUMED`, `RELEASED`, `EXPIRED`.
- **Identificadores:** UUID v4; `orderNumber` entero autoincremental legible para el cliente.

Reglas del MVP y de la importación

- **Reservas:** `Reservation` y `ReservationItem` existen desde la Fase 1; la reserva es la unidad de retención y de expiración también sin Redis, y la Fase 2 solo añade Redis encima, sin cambios de esquema.
- **Vendedores:** en la Fase 1 `POST /seller-profile` crea el perfil como `APPROVED`; es solo una simplificación del MVP. El estado `PENDING` y la moderación llegan con la administración (Fase 3).
- **Administradores:** las cuentas `ADMIN` se crean por seed solo con ese rol; el checkout exige `CUSTOMER`.
- **Importación:** los precios de DummyJSON y FakeStoreAPI se tratan como EUR sin conversión (datos de demostración) y cada producto recibe una sola categoría, mapeada desde la de origen. Open Food Facts se aplaza; cuando se importe, sus precios serán generados de forma determinista.
- **AuditLog:** la tabla y sus escrituras llegan en la Fase 3; en la Fase 1 los eventos de seguridad van al log de la aplicación.

Invariantes

Id	Invariante
I1	`0 <= reserved <= onHand` (CHECK)
I2	`onHand` = suma de `delta` de los movimientos del producto
I3	`reserved` = suma de las líneas de reservas `ACTIVE` del producto
I4	Un pedido tiene como máximo una reserva; si está `PENDING_PAYMENT`, su reserva está `ACTIVE`
I5	Como máximo un pago vivo por pedido
I6	`lineTotal = unitPrice × quantity` y `total = subtotal + shipping`
I7	Un evento de webhook `(provider, eventId)` se procesa una sola vez

Id	Invariante
I8	Una reserva sale de `ACTIVE` una sola vez

Resumen para Prisma (sin `schema.prisma` completo en esta etapa): tablas en `snake_case` plural con `@@map` y columnas en `camelCase`; enums nativos; `@db.Uuid` y `@db.Timestamptz(3)`; borrado lógico con `deletedAt` solo en `User` y `Product`; claves foráneas `Restrict` en todo lo que toca dinero o historial. CHECK, índices parciales, `pg_trgm` y triggers de solo inserción no se expresan en Prisma y van como SQL manual dentro de las migraciones.

5. Inventario cuantificado de módulos

La arquitectura objetivo tiene 7 módulos, 20 controllers, 55 endpoints HTTP, 34 services y 16 entidades. Son objetivos de planificación, no código ya implementado. La v1 implementa 33 de los 34 services porque StripeGatewayService queda fuera (A9).

Módulo	Controllers	Endpoints	Services	Entidades	Responsabilidad
Auth & Users	3	12	6	4 (`User`, `UserRole`, `SellerProfile`, `RefreshTokenSession`)	Registro, login, sesiones con rotación, perfil, perfil de vendedor, RBAC
Catalog	3	12	5	2 (`Category`, `Product`)	Productos, categorías, búsqueda, CRUD del vendedor, importación
Inventory & Reservations	3	7	4	4 (`Inventory`, `InventoryMovement`, `Reservation`, `ReservationItem`)	Stock autoritativo, movimientos, reservas atómicas, caché Redis, reconciliación
Orders & Checkout	3	7	5	2 (`Order`, `OrderItem`)	Cotización, checkout idempotente, pedidos, máquina de estados, cumplimiento por línea
Payments & Webhooks	3	4	6	3 (`Payment`, `WebhookEvent`, `IdempotencyKey`)	Pagos, pasarela mock, adaptador Stripe, webhooks firmados e idempotentes

Módulo	Controllers	Endpoints	Services	Entidades	Responsabilidad
Administration & Audit	3	9	4	1 (`AuditLog`)	Gestión de usuarios y operaciones, escritura y consulta de auditoría
Notifications & Background Jobs	2	4	4	0	Colas BullMQ, notificaciones simuladas, health checks, planificación del reconciliador
Total	**20**	**55**	**34**	**16**	

Criterio de conteo:

- Un endpoint es una combinación de método HTTP y ruta, y se cuenta una sola vez en el módulo propietario del recurso. Los endpoints administrativos van en el módulo que administran (por ejemplo, la reconciliación de stock cuenta en Inventory), salvo usuarios, operaciones y auditoría, que van en Administration & Audit; los dos health checks cuentan en Notifications & Background Jobs.
- Un controller agrupa endpoints de un mismo recurso y rol de acceso; con Fastify es un plugin de rutas.
- Un service es una clase o módulo de funciones con lógica de negocio o de integración; no se cuentan hooks, repositorios ni los procesadores de las colas del worker.
- Reparto por fase de los 55 endpoints: 22 en la Fase 1, 3 en la Fase 2 y 30 en la Fase 3 (los 19 aplazados de la Fase 1 más 11 propios de la Fase 3, todos completos antes de G3).

6. Diseño conceptual de seguridad, pagos y reservas

Autenticación y autorización

- **Access token:** JWT de 15 minutos con claims mínimos (`sub`, `roles`, `sid`), mantenido solo en memoria del cliente y enviado como `Bearer`.
- **Refresh token:** valor opaco de 256 bits; en base de datos solo se guarda su hash SHA-256. Cookie `HttpOnly`, `Secure` en producción, `SameSite=Strict`, `Path=/api/v1/auth`, 7 días.
- **Rotación y revocación:** cada refresh invalida el token usado y emite otro en la misma familia; un token reutilizado revoca toda la familia y se registra (en el log de la aplicación durante la Fase 1 y en AuditLog desde la Fase 3); el logout revoca la familia. Redis acelera la comprobación desde la Fase 2, pero PostgreSQL es autoritativo.

- **Roles y propiedad:** `CUSTOMER`, `SELLER` y `ADMIN`; la propiedad del recurso (`customerId` del pedido, `sellerId` del producto) se comprueba siempre en el servidor, y las operaciones sensibles revalidan los roles contra la base de datos. Las cuentas ADMIN se crean por seed solo con ese rol y el checkout exige CUSTOMER, de modo que un administrador no compra; un vendedor tampoco puede comprar sus propios productos (409).
- **CSRF:** solo `/auth/refresh` y `/auth/logout` usan la cookie; se protegen con `SameSite=Strict`, verificación de `Origin` y una cabecera personalizada. El resto usa `Bearer`, que el navegador no adjunta solo.
- **Límites de tasa y validación:** contadores de ventana para login, registro, checkout, webhooks y lecturas, en memoria del proceso durante la Fase 1 (válido con una sola instancia) y en Redis desde la Fase 2 (429 con `Retry-After`); validación Zod en cada frontera; secretos solo en variables de entorno de la plataforma.

Pagos y webhooks

- **Pasarela:** un puerto `PaymentGateway` con un adaptador mock (por defecto) y un adaptador Stripe Test Mode (opcional, posterior a la v1). El mock no es un sistema financiero real y la aplicación nunca recibe ni guarda datos de tarjeta.
- **Webhooks:** se firma `timestamp.cuerpo` con HMAC SHA-256 sobre el cuerpo crudo, se acepta una tolerancia de 5 minutos y la comparación es en tiempo constante. Cada evento válido se inserta en una bandeja de entrada con `UNIQUE(provider, eventId)`; se procesa en una transacción con compare-and-set; un duplicado devuelve 200 sin efecto.

Evento	Resultado
Pago exitoso a tiempo	Pago `SUCCEEDED`, pedido `PAID`, reserva `CONSUMED` y stock descontado de forma definitiva
Pago rechazado	Pago `FAILED`; el pedido sigue pendiente (máximo 3 intentos) y la reserva activa
Cancelación o expiración	Pedido `CANCELLED` o `EXPIRED`, pago `CANCELLED` y reserva liberada
Pago tardío (pedido ya expirado)	Pago `REFUNDED` (simulado); el pedido no se reactiva

Reservas atómicas con Redis y Lua

1. El cliente inicia el checkout con una `Idempotency-Key`; la API valida los artículos y recalcula los precios en PostgreSQL.
2. Un script Lua descuenta la disponibilidad de todas las líneas o de ninguna, de forma idempotente por `reservationId`, y registra la reserva con su caducidad.
3. Una transacción de PostgreSQL crea el pedido `PENDING_PAYMENT`, sus líneas y la reserva `ACTIVE`, y aplica un `UPDATE` condicional (`onHand - reserved >= q`) por línea en orden ascendente de producto. Si PostgreSQL rechaza, se compensa en Redis.
4. Se encola un trabajo de BullMQ con 10 minutos de retraso; un barrido periódico cubre su posible pérdida.
5. El pago exitoso consume la reserva; el rechazo definitivo, la cancelación o la expiración la liberan.
6. La expiración pasa la reserva de `ACTIVE` a `EXPIRED` con compare-and-set en PostgreSQL, devuelve `reserved` y libera la reserva en Redis de forma idempotente.

Fase 1 (sin Redis): se omiten el paso 2, el paso 4 y la compensación en Redis. La transacción de PostgreSQL con la reserva `ACTIVE` y el `UPDATE` condicional es el único control, y la expiración la ejecuta un barrido con `node-cron` en la propia API, cada 30 segundos y al arrancar. La Fase 2 añade Redis delante sin cambiar el esquema.

Límites: Lua es atómico solo dentro de Redis; no existe una transacción distribuida con PostgreSQL, y entre ambos hay una ventana en la que uno puede haber cambiado y el otro no. Se gestiona con tres mecanismos: PostgreSQL como árbitro final (`UPDATE` condicional y CHECK), compensaciones idempotentes y una reconciliación periódica en la que PostgreSQL gana.

Riesgo	Tratamiento
Duplicados (doble clic, webhook repetido)	`Idempotency-Key`, `UNIQUE` de evento y compare-and-set
Pago y expiración simultáneos	Ambos compiten por el mismo compare-and-set sobre la reserva `ACTIVE`; gana el primero
Redis reiniciado o no disponible	El checkout responde 503 controlado; la reconciliación recarga los contadores desde PostgreSQL y recrea las reservas `ACTIVE`
Reserva en Redis sin fila en PostgreSQL	El reconciliador la libera pasados 2 minutos
Worker retrasado o caído	Los trabajos persisten y un barrido cada 60 s recupera lo vencido; es una degradación (stock retenido más tiempo), no una incorrección

Riesgo	Tratamiento
Pago tardío	Reembolso simulado; el pedido expirado no se reactiva

7. Objetivos de calidad y riesgos

Todas las cifras son estimaciones de diseño; no hay ningún resultado medido todavía. Los valores reales solo se conocerán tras las pruebas de carga de la Fase 3 y los objetivos se ajustarán con esa evidencia. «Laboratorio» significa API, PostgreSQL y Redis locales con una sola instancia; «producción» significa planes gratuitos, latencia de red y sin contar arranques en frío.

Operación	Lab p95 (ms)	Lab p99 (ms)	Producción p95 (ms)
Listado de catálogo	80	150	400
Ficha de producto	50	100	300
Escritura de catálogo	150	300	600
Inicio de checkout (incluye reserva)	200	400	800
Script Lua de reserva (medido en servidor)	8	20	100
Confirmación de pedido (webhook → `PAID`)	150	300	600
Webhook duplicado (200 sin efecto)	40	80	300

Tasas de error: respuestas 5xx por debajo del 0,5% en laboratorio y del 1% en producción; los rechazos de negocio esperados (por ejemplo `INSUFFICIENT_STOCK`) no cuentan como error.

Pruebas de concurrencia: 50 checkouts simultáneos sobre stock 10 (Fase 1) y 200 sobre stock 50 (Fase 2), repetidas 10 veces seguidas; doble clic con la misma `Idempotency-Key`; el mismo webhook enviado 10 veces; reinicio de Redis y caída del worker a mitad de carga.

Criterio	Objetivo
Seguridad	Revisión contra OWASP Top 10; pruebas de reutilización de refresh, acceso a recursos ajenos, firma inválida, límites de tasa y CSRF; ningún secreto en el repositorio
Disponibilidad	99% mensual para la demo, sin garantía; ante un fallo de Redis, un 503 controlado es preferible a datos incorrectos
Consistencia	0 ventas por encima del stock; invariantes I1–I8 verificados tras cada prueba; divergencia Redis–PostgreSQL detectada en ⩽ 5 min
Recuperación	Expiración procesada con p95 ⩽ 30 s; reservas vencidas procesadas ⩽ 60 s tras reiniciar el worker; Redis reconstruido desde PostgreSQL en ⩽ 5 min

Riesgos principales

Riesgo	Prob.	Impacto	Mitigación
El polling de BullMQ agota los comandos del plan gratuito de Redis	Alta	Alto	Medirlo pronto, ajustar intervalos o usar Redis local; alternativa pg-boss
El servicio gratuito de Render duerme y retrasa las expiraciones	Alta	Medio	Barrido al arrancar y periódico; documentar la degradación
Divergencia entre Redis y PostgreSQL	Media	Alto	Reconciliación periódica; PostgreSQL manda; pruebas de caos
Pausa o límites del plan gratuito de Supabase	Media	Alto	Copias con `pg_dump`, seed reproducible y migraciones versionadas
Las cookies entre dominios no funcionan	Media	Alto	Proxy de mismo origen, probado desde el primer día del frontend
Webhooks duplicados o fuera de orden	Alta	Medio	Bandeja de entrada, compare-and-set y estado `IGNORED`
Deadlocks en reservas multilínea	Media	Medio	Orden ascendente por producto y reintentos acotados

Riesgo	Prob.	Impacto	Mitigación
Crecimiento del alcance	Alta	Alto	Fases con puertas de salida y lista de funciones aplazadas

8. Decisiones arquitectónicas definitivas

Estas decisiones quedan fijadas para las partes 2 y 3; cambiarlas exige reabrir el diseño.

ID	Decisión	Alternativa descartada
D1	Monolito modular con worker separable	Microservicios
D2	Fastify con plugins por módulo	NestJS
D3	PostgreSQL decide; Redis admite y acelera	Redis como fuente de verdad
D4	`Inventory` aparte con `onHand` y `reserved`, más libro de movimientos	Contador dentro de `Product`
D5	Roles como enum en `UserRole`, sin tabla `Role`	Permisos dinámicos
D6	JWT de acceso de 15 min en memoria; refresh opaco en cookie con rotación por familias	Refresh en `localStorage`; JWT de refresh sin estado
D7	Proxy de mismo origen mediante `rewrites` de Next.js	Cookies entre dominios
D8	Dinero en céntimos enteros y una sola moneda (EUR)	`Decimal` y multimoneda
D9	Una categoría por producto	Tabla `ProductCategory`
D10	Carrito solo en el cliente, validado en el servidor	Carrito persistido en servidor
D11	Pasarela mock por defecto con puertos y adaptadores; Stripe Test Mode opcional	Stripe obligatorio
D12	Webhooks con bandeja de entrada, `UNIQUE(provider, eventId)` y HMAC sobre el cuerpo crudo	Deduplicar solo en memoria

ID	Decisión	Alternativa descartada
D13	BullMQ con barrido de seguridad independiente; worker `embedded` en el plan gratuito	Servicio de worker dedicado obligatorio
D14	Un pedido expirado no se reactiva; el pago tardío se reembolsa (simulado)	Intentar re-reservar
D15	Paginación por cursor	`OFFSET`

Decisiones añadidas al cerrar la Parte 1

ID	Decisión	Alternativa descartada
D16	Límites de tasa en memoria durante la Fase 1 y en Redis desde la Fase 2; con una sola instancia los contadores en memoria bastan, aunque se pierden al reiniciar	Redis desde la Fase 1
D17	`SellerProfile` se crea `APPROVED` en la Fase 1, solo como simplificación del MVP; la moderación llega en la Fase 3	Aprobación manual desde la Fase 1
D18	Fase 1 limitada a 22 endpoints núcleo; los 19 restantes se completan antes de G3; `AuditLog` y panel de vendedor pasan a la Fase 3	Los 41 endpoints en la Fase 1
D19	`Reservation` y `ReservationItem` existen desde la Fase 1; Redis se añade en la Fase 2 sin cambios de esquema	Crear las tablas de reserva en la Fase 2
D20	Cuentas `ADMIN` por seed solo con ese rol; el checkout exige `CUSTOMER`; límite blando de 3 pedidos pendientes; un vendedor no compra sus propios productos	Regla especial «el administrador no compra»
D21	Los precios importados se tratan como EUR sin conversión; una categoría por producto mapeada; Open Food Facts se aplaza con precios deterministas generados	Conversión de moneda real
D22	La v1 implementa 33 de los 34 services; `StripeGatewayService` queda como objetivo posterior	Implementar Stripe en la v1

ID	Decisión	Alternativa descartada
D23	Precisiones: el rewrite de D7 conserva el prefijo `/api/v1` y D13 incluye ejecutar el barrido también al arrancar la API	—

Aprobaciones A1–A9 (cerradas)

Aprobadas el 2026-10-11 con los ajustes indicados en cada línea. No queda ninguna decisión de diseño pendiente; solo verificaciones externas (límites de planes gratuitos, licencias de las APIs de datos y versiones LTS).

- [x] ~~**A1 · Fastify en JavaScript, sin NestJS (D2).** Menos estructura impuesta por el framework y más disciplina propia, a cambio de usar JavaScript puro sin decoradores ni transpilado.~~
- [x] ~~**A2 · Redis como control de admisión (D3).** PostgreSQL solo ya evita la sobreventa; Redis se mantiene por valor didáctico y llega en la Fase 2, a costa de complejidad y de depender de los límites de un plan gratuito.~~
- [x] ~~**A3 · Worker `embedded` en el plan gratuito (D13).** Si la instancia duerme, las expiraciones se retrasan; en la Fase 1 el barrido corre en la API con `node-cron`, cada 30 s y al arrancar.~~
- [x] ~~**A4 · Proxy de mismo origen (D7).** Añade un salto de red; el rewrite conserva el prefijo `/api/v1` y hay que comprobar que `rewrites` funciona hacia un dominio externo y tolera el arranque en frío de Render.~~
- [x] ~~**A5 · Simplificaciones (D8, D9, D10).** Una moneda, una categoría por producto y carrito sin sincronización entre dispositivos; los precios importados se tratan como EUR sin conversión.~~
- [x] ~~**A6 · Redis caído (sección 6).** Aplica desde la Fase 2: el checkout responde 503 en lugar de degradar a la ruta de solo PostgreSQL, que queda como mejora futura.~~
- [x] ~~**A7 · Pago tardío (D14).** Se reembolsa de forma simulada y el pedido expirado no se reactiva.~~
- [x] ~~**A8 · Reglas de negocio.** Máximo de 3 pedidos pendientes por usuario (límite blando: es una medida anti-acaparamiento y la integridad del stock la garantiza la reserva); un vendedor no compra sus propios productos; las cuentas `ADMIN` se crean por seed solo con ese rol y el checkout exige `CUSTOMER`.~~
- [x] ~~**A9 · Stripe fuera de la v1 (D11).** Solo la pasarela mock; se implementan 33 de los 34 services.~~

HANDOFF PARTE 1

Copiar el bloque siguiente al inicio de la Parte 2. Es la fuente de verdad: ningún nombre, estado ni cifra cambia sin reabrir el diseño.

```
HANDOFF PARTE 1 (DEFINITIVO) · MarketFlow · System Data Sheet v1.0 · 2026-10-
11
Fuente de verdad para la Parte 2. Aprobadas A1-A9 y D1-D23. Ningun nombre,
estado ni cifra
cambia sin reabrir el diseno.

PRODUCTO
Marketplace multi-vendor B2C/C2C: catalogo, vendedores, carrito, checkout con
reserva de 10 min,
pedidos con precios historicos, pagos simulados, webhooks idempotentes,
expiracion automatica.
DummyJSON y FakeStoreAPI solo se usan para importar; la app funciona sin
ellas.

STACK
- JavaScript moderno ES2022+ con ES modules, SIN TypeScript (.js, .jsx, .mjs;
sin tsconfig ni
  typecheck; CI = lint + tests).
- Next.js App Router + React (JSX), Tailwind, shadcn/ui.
- Fastify con plugins por modulo (prefijo /api/v1; hooks preHandler para auth
y roles;
  fastify.decorate para services; setErrorHandler; cuerpo crudo de webhooks
con parser propio).
  Zod valida en ejecucion; packages/contracts exporta solo esquemas. No
NestJS.
- PostgreSQL + Prisma; Redis y BullMQ desde las Fases 2 y 3; node-cron en la
Fase 1.
- Vitest con fastify.inject, Supertest, Testcontainers, Playwright, k6; pnpm,
Docker, Actions.
- Hosting: Vercel (web), Render (API, worker embedded), Supabase, Upstash. El
navegador llama a
  /api/* en Vercel, que reescribe a Render conservando el prefijo. Limites
SIN VERIFICAR.
- Monorepo: apps/web, apps/api (main.js, main.worker.js),
packages/{db,contracts,config}.

ENTIDADES (16)
User, UserRole, SellerProfile, RefreshTokenSession, Category, Product,
Inventory,
InventoryMovement, Reservation, ReservationItem, Order, OrderItem, Payment,
WebhookEvent,
```

```
IdempotencyKey, AuditLog (tabla y escrituras en la Fase 3).
Dinero en centimos enteros (EUR); timestamptz UTC; UUID v4 + orderNumber;
tablas snake_case,
columnas camelCase; deletedAt solo en User y Product; FK Restrict en dinero e
historial; CHECK,
indices parciales, pg_trgm y triggers append-only como SQL manual en las
migraciones.
Reservation y ReservationItem existen desde la Fase 1.
ENUMS: Role CUSTOMER SELLER ADMIN · UserStatus ACTIVE SUSPENDED ·
SellerStatus PENDING APPROVED
SUSPENDED · ProductStatus DRAFT ACTIVE ARCHIVED · ProductSource MANUAL
DUMMYJSON FAKESTORE
OPENFOODFACTS · InventoryMovementType INITIAL ADJUSTMENT SALE RETURN ·
ReservationStatus ACTIVE
CONSUMED RELEASED EXPIRED · OrderStatus PENDING_PAYMENT PAID CANCELLED
EXPIRED COMPLETED ·
FulfillmentStatus PENDING SHIPPED DELIVERED CANCELLED · PaymentProvider MOCK
STRIPE ·
PaymentStatus PENDING REQUIRES_ACTION SUCCEEDED FAILED CANCELLED REFUNDED ·
WebhookStatus
RECEIVED PROCESSED FAILED IGNORED · IdempotencyStatus IN_PROGRESS COMPLETED ·
AuditActorType
USER SYSTEM WEBHOOK
INVARIANTES: I1 0<=reserved<=onHand · I2 onHand=SUM(delta) · I3
reserved=SUM(lineas de
reservas ACTIVE) · I4 una reserva por pedido; PENDING_PAYMENT implica ACTIVE
· I5 un pago vivo
por pedido · I6 totales coherentes · I7 webhook unico (provider,eventId) · I8
una reserva sale
de ACTIVE una sola vez. Transiciones con compare-and-set: Order
PENDING_PAYMENT->PAID|CANCELLED|
EXPIRED, PAID->COMPLETED|CANCELLED; Payment PENDING->REQUIRES_ACTION-
>SUCCEEDED|FAILED|
CANCELLED, SUCCEEDED->REFUNDED.

MODULOS: 7 modulos, 20 controllers (plugins de rutas), 55 endpoints, 34
services (33 en v1),
16 entidades. Controllers/endpoints/services/entidades: Auth&Users 3/12/6/4 ·
Catalog 3/12/5/2 ·
Inventory&Reservations 3/7/4/4 · Orders&Checkout 3/7/5/2 · Payments&Webhooks
3/4/6/3 ·
Admin&Audit 3/9/4/1 · Notifications&Jobs 2/4/4/0. Endpoints por fase: 22 / 3
/ 30.

FASE 1 = 22 ENDPOINTS
Auth: register, login, refresh, logout, GET /users/me, POST /seller-profile
```

```
(crea APPROVED).
Catalogo: GET /products, GET /products/:slug, GET /categories.
Vendedor: GET, POST y PATCH /seller/products; POST
/seller/inventory/:productId/adjustments.
Pedidos: POST /checkout, GET /orders, GET /orders/:id, POST
/orders/:id/cancel.
Pagos: POST /orders/:id/payments, POST /mock-gateway/payments/:id/confirm,
POST /webhooks/payments/:provider. Operacion: GET /health/live y
/health/ready.
Aplazado (19, antes de G3): sesiones y logout-all, PATCH /users/me, perfil de
vendedor,
CRUD de categorias, GET /categories/:slug, ver/borrar producto propio,
disponibilidad,
inventario del vendedor y movimientos, cotizacion, pedidos de vendedor, GET
/payments/:id.
TAREAS: 1 repo+CI · 2 API minima · 3 Prisma auth y catalogo · 4 registro,
login, me,
seller-profile · 5 refresh y logout · 6 limites de tasa en memoria · 7
importacion y catalogo
publico · 8 productos del vendedor, inventario, pedidos y UPDATE condicional
· 9 checkout,
reserva y expiracion · 10 pago mock y webhook · 11 frontend minimo · 12
puerta G1.
PUERTAS: G1 50 checkouts sobre stock 10 -> 10 reservas (x10 repeticiones) ·
G2 200 sobre stock
50 -> 50 con reinicio de Redis · G3 expiracion p95 <= 30 s y checklist.

CONCURRENCIA
PostgreSQL decide (UPDATE condicional onHand-reserved>=q + CHECK; lineas en
orden de producto).
Fase 1: reserva ACTIVE en PostgreSQL y barrido node-cron (30 s y al
arrancar). Fase 2: Redis +
Lua todo o nada, idempotente por reservationId; Lua no es transaccion
distribuida: compensacion
e reconciliacion (5 min, PostgreSQL gana); Redis caido -> 503. Fase 3: BullMQ
retrasado 10 min
+ barrido cada 60 s.

SEGURIDAD Y PAGOS
JWT 15 min en memoria; refresh opaco 256 bits (hash en BD), cookie HttpOnly,
Secure en
produccion, SameSite=Strict, Path=/api/v1/auth, 7 dias; rotacion por familia;
reutilizacion
revoca la familia y va al log (AuditLog en Fase 3). Limites de tasa en
memoria (Fase 1) y Redis
(Fase 2). Mock por defecto; Stripe fuera de v1; sin datos de tarjeta.
```

```
Webhook: HMAC SHA-256 de
timestamp.cuerpo crudo, tolerancia 5 min, UNIQUE(provider,eventId), CAS en
transaccion. Pago
tardio -> REFUNDED simulado; pedido expirado no se reactiva.
REGLAS: 3 intentos de pago; 10 unidades por linea; 20 lineas; 3 pedidos
pendientes (blando);
vendedor no compra lo suyo; ADMIN por seed solo con ADMIN; checkout exige
CUSTOMER.
IMPORTACION: precios como EUR sin conversion; una categoria por producto;
Open Food Facts
aplazado.

CALIDAD (estimaciones, NO medidas)
Lab p95/p99 ms: catalogo 80/150, ficha 50/100, escritura 150/300, checkout
200/400, Lua 8/20,
confirmacion 150/300. 5xx < 0,5% lab y < 1% produccion. 0 ventas sobre el
stock.

PENDIENTE DE VERIFICAR
Limites de Upstash, Render, Supabase y Vercel; licencias de DummyJSON,
FakeStoreAPI y Open Food
Facts; soporte de shadcn/ui sin TypeScript; versiones LTS de Node, Next.js,
Fastify y Prisma.
```